

為 gfx1100 用戶追回軟件算力：GIGABYTE W7800 48G 單卡跑通 AMD 原版 Quark INT4，實測、三項原創補丁、一個尚未定位的 10% 差距

測試平台：GIGABYTE Radeon™ PRO W7800 AI TOP 48G (單卡) gfx1100 / RDNA3 | 70 CU | ROCm 7.2.4
| Ubuntu 24.04 | Ryzen 5 7500F | 30 GB RAM 2026-09-15

○、實測成績：DSH 真實 agent 路徑

本節所有數字均直接抽取自引擎日誌，並非實驗室短提示評測。

真實 agent 工作負載 (DSH，上下文約 20K)

指標	值
decode 吞吐	62.11 t/s
樣本數	11 (59.55 / 62.11 / 63.94 / 59.49 / 52.37 / 59.95 / 67.80 / 78.91 / 71.91 / 76.56，另有 1 個 9.35 離群值)
accept length	2.87 – 3.70
同一台機器、短提示單流	85.05 t/s

⇒ 兩者唯一的變數是上下文深度 (full token 20,004 – 20,944)。這就是 agent 場景的真實成本。

複驗 (2026-09-15，HiCache 8 + chunk 8192)

指標	值
decode 吞吐	61.5 t/s (n=23，full token 中位數 24,299)
p90	80.4 t/s
accept length 中位數	3.40

⇒ 與 09-14 的 62.11 t/s 一致，並無退步。(用戶主觀感受的 74 t/s 屬較短上下文。)

生產環境實績

項	值
decode 中位數 (投產時)	57–60 t/s
KV cache 命中率	65 – 77%
曾經承擔	11 個 subagent 並行審計
split-K 512 與 1024 之 A/B	70 對 61 t/s

這些數字在任何 gfx1100 單卡用戶的環境中皆可復現。

一、我們是誰、在什麼卡上做

我們只有一張 GIGABYTE Radeon™ PRO W7800 AI TOP 48G (gfx1100、**70 CU**)。

單卡這一事實至關重要，因為它決定了我們所有結論的邊界：

	我們	論壇其他前輩
卡	W7800 48G ×1 (70 CU)	7900 XTX ×2 (各 96 CU) / 4090 48G 改裝版 ×1 / 4080S ×2
總 VRAM	48 GB	48 GB (雙 7900XTX 24+24) / 48 GB
TP	1	2 (雙卡)
卡片架構	RDNA3 gfx1100	7900XTX 同為 gfx1100 但 96 CU ; 4090 為 NVIDIA

⇒ 本文出現的任何數字，其前提皆為「單卡、70 CU、gfx1100」。

W7900 (96 CU)、7900 XTX (96 CU) 與 W7800 (70 CU) 雖然同屬 **gfx1100**，但 CU 數不同，吞吐量不可直接套用。我們先前見到有材料把 CU 數寫成 96 —— 那是 W7900 的數字，我們這張是 **70** (amd-smi 實測)。

我們的目標很簡單：把 **gfx1100** 用戶所能運用的軟件算力，逐分逐毫地榨取出來，而且每一步都可復現。

二、主軸線：這條路的形成過程

起點：一句提示把我們推了進來

2026-09-05 我們試用 SGLang 官方版，結論為「已列名，但尚未就緒」(FP8 只有 0.62 t/s)。正想放棄的時候，**Terry** 在我們的報告下面留下一句：

「你可以嘗試下 SGLang，論壇有帖子，體驗會好很多」

一句提示，我們投入了十天。沿著他指出的方向，我們找到論壇前人已經鋪好的路 (見文末致謝)，最後以社群的 gfx 1100-support fork 完成三引擎完整對比，才定下生產配置。

⇒ **這是本文最想說明的一件事：這條路是社群鋪出來的，我們只是接續前行。**

中段：找到真正的病灶

我們做過許多無效的優化，代價最高的一課是：**有一整輪優化耗在一個穩態佔比 0% 的 kernel 上** —— 那個檔案佔 46 % 的代碼量，卻只佔 4.59% 的 GPU 時間；真正的熱核佔 61.14%。

質量集中處 ≠ 能量集中處。

這句話後來成為我們所有優化的第一道關卡：**動手之前先量能量分佈。**

轉折：接入 AMD 原版模型

AMD 在 HuggingFace 放出 **amd/Qwen3.8-27B-Quark-AWQ-INT4-W4A16**，但它在 RDNA3 上**無法運行** —— SGLang 的 Quark 模組只實作了三種 scheme，純 W4A16 沒有；**quark/weights.py** 甚至直接註明「Scheme didn't allocate the parameter (e.g. W4A16)」。

我們撰寫了轉換器把它接入。**這是我們第一件原創作品** (詳見第三節)。

關鍵一段：INT4 之後，速度是怎樣追回來的

接通 AMD 原版模型之後，我們面對第二個問題：模型能跑，但速度不夠。

我們寫了一個 Python shim (`rdna_shim/rdna_lmhead_int8.py`)，經 `PYTHONPATH` 注入，在 vendored 樹只加三行攔截 `_compute_lm_head`。它不改 kernel，而是換路線：lm_head 是 248320×5120 的 bf16 矩陣 (2.543 GB)，每步被讀四次 (verify 一次 M=4、draft 三次 M=1)，合共約 13.9 ms/步、佔步時約 20%；M=1 時完全是記憶體綁死，位元寬直接決定成本。改用 INT4 之後每次只讀 **0.661 GB**。

階段	引擎吞吐 (5K 上下文、temp 0)	相對基準
bf16 lm_head (原狀)	57.12 t/s	—
int8 lm_head (中間步驟)	60.43 t/s	+5.8%
int4 lm_head (最終)	68.38 t/s	+19.7%

輸出 MD5 三者完全相同，accept length 4.0 不變，U+FFFD 為 0。兩次獨立量測得 68.41 / 68.38。

在此之上，我們又改了 kernel 本體兩處：WMMA 門檻由 M≥16 下移至 M≥9 && N≥2048 (M=9 GEMM 246.7 → 185.9 μs，-24.6%)，以及 B2 編譯期模板化 (M=4 -9.3%、M=5 -8.7%、M=8 -14.2%)。端到端 **68.38 → 72.84 t/s**，累計對 bf16 基準 +27.5%，輸出與基準逐字節相同。

速度追回來之後，我們再把底盤逐件接回：MTP-3 投機解碼 (09-11 由 DFLASH 折返後重新接上)、CUDA graph、HiCache (命中率 65% → 77%，09-14 重新上線)，最後是 09-14 傍晚的 DFLASH8 + HiCache8 (單流 92.57 t/s)。

⇒ 這一段的代價，同樣如實記錄：

時間	失敗與代價
09-11 13:20	投機門檻下移 M≥2 雖然修復了確定性，代價是吞吐 -15.2% (88.9 t/s)。交易為負，已放棄並還原。
09-11 13:20	draft 量化 v3：體積由 3.85 GB 降到 1.743 GB，但 accept rate 掉到 0.00、吞吐 15.7 t/s (-85%)
09-11 14:14	事故：INT4 一度被洗成 W4A16 —— <code>exp_config.py</code> 的 REF 寫死了 W4A16 配方，13:31 起所有受控測試全部跑錯模型
09-11 22:36	差一點誤報成功：見到 HYBRID 啟動與 84.9 t/s，真相是 HYBRID 失敗後系統 fallback 到 DFLASH；靠 <code>/proc/<pid>/cmdline</code> 硬驗證才攔截下來
09-11 22:36	字串拼接改配置令反斜線續行斷裂，生產停擺 3.5 分鐘
09-13	P29-P52 工藝戰役：18 個假說全部否認 (全部落在 ±0~2%)，隨後證明整場戰役打在那個只佔 4.59% GPU 時間的 kernel 上

⇒ 這就是「質量集中處 ≠ 能量集中處」的代價，也是我們把它列為第一道關卡的原因。

現在：把差距交代清楚

接入之後，我們量到自身的服務品質低於 AMD 公布的數字 (PPL 高一成、GSM8K 低七分)。我們排除了六個嫌疑，但機制仍未定位 —— 此點我們在第四節誠實交代。

三、我們原創的三項成果

3.1 Quark W4A16 → SGLang GPTQ 轉換器 (已開源)

<https://github.com/lawsirlawsir-png/rdna3-quark-w4a16>

原意：讓 gfx1100 用戶不必等待 SGLang 補上 W4A16 scheme，當日即可運行 AMD 原版模型。

實質作用：把 Quark 的 N-packed `[K, N/8]` 佈局重新打包成 GPTQ 的 K-packed `[K/8, N]`，權重未改動任何一個位元。

技術核心 (亦是我們耗時最久之處)：Quark 在 `pack_method="reorder"` 之下，每個 int32 內的 8 個 4-bit 值並非順序排列，而是按 `order_map = [0,2,4,6,1,3,5,7]` 重排。忽略這一步，整個模型將輸出亂碼。

我們當初正是遺漏了這一步，以數值掃描猜測數小時之久；最終閱讀官方 ``quark/torch/utils/pack.py`` 之後即得解。

⇒ 教訓：凡涉及外部格式，先讀官方實作，不可依靠試探。此後成為我們的鐵律。

無損驗證 (三條獨立證據)：

檢驗	結果
逐元素反量化比對 (5 層，含 N=48 的極瘦層)	max 絕對差 = 0.000000e+00
全層覆蓋	496 個量化層 + 703 個普通層
對 BF16 原模型的忠實度 (96 個 in_proj)	cos 最小 0.9871、中位數 0.9905

3.2 N-aware K-split (小 M 專用，-7.9%)

Commit：0d2e013 perf(rdna3-wmma): opt-in N-aware k_split for small-M; -7.9% on M=16 verify GEMM across all 6 p
roduction shapes, fp64-identical accuracy; default path unchanged

原意：上游的 `compute_wmma_k_split()` 只看 K、不看 N，因此在 $K \in \{5120, 6144, 17408\}$ 時一律返回 4。但在我們的生產形狀中，N=5120 佔權重的 23.6%，在 k=4 之下嚴重佔用不足。

實質作用：改為 N-aware 規則 —— 選取「令 block 數達到約 5120 的最小 k」。在我們六個生產形狀上逐一掃描後：

形狀	k=4	k=8	k=16	k=32	最優
5120 × 17408	0.22503	0.22087	0.22612	0.25149	k=8
17408 × 5120	0.26621	0.26696	0.22434	0.22263	k=32
5120 × 10240	0.16138	0.13935	0.14128	0.15573	k=8
6144 × 5120	0.10095	0.10367	0.09191	0.09835	k=16
5120 × 6144	0.09661	0.09436	0.09122	0.09996	k=16
5120 × 12288	0.17250	0.16021	0.16392	0.18090	k=8

規則在 6 個形狀中有 5 個命中實測最優；唯一例外為 17408×5120，差 0.77% (按真實層混合加權後更小)。

最重要的一點：準確度在 fp64 之下逐位元相同，且預設路徑完全不變 (須開啟 `SGL_WMMMA_KSPLIT_AUTO=1` 方生效)。我們其後實測關閉它：PPL 9.7296 對開啟時 9.7297，差 -0.00% ⇒ 這個效能補丁在數值上是忠實的。

3.3 其他原創改動 (逐項交代原意與作用)

Commit	原意	實質作用
--------	----	------

c2d509e	小 M WMMA 門檻下移至 M>=9	覆蓋 decode verify step (生產實測 80% 為 M=4 , 其餘為 M=8)
9af1d8d	GPTQ kernel 編譯期模板化	Σa 預算與 z 修正改為 每 group 一次 , 減少重複計算
b10a8ab	正確性修正	把不規則 (NGRAM) 樹排除於 mask-less unified verify kernel 之外
01d62d6	生產樹入版控	GPTQ M_COUNT 5/6/7、lm_head 低位寬
1f1ba80	WMMA M=16 瓶頸研究	env-gated 量測變體, 預設路徑不變
23dd307	建置整潔	把量測變體 排除於生產 build 之外
6c5409c	建置整潔	停止追蹤 21 個由 hipify 產生的 .hip 檔
005ad71	修錯	CU 數 96 → 70 (96 為 W7900 ; 本卡為 70)
f736a74	首次入版控	2026-09-08/09 的 gfx1100 生產補丁

另外兩項不在 git 之內、但同樣屬於我們原創的：

- `prod/rdna_shim/rdna_lmhead_int8.py` —— **INT4 LM head**, 只在小 batch 觸發
- 我們整套量測方法論 (見第五節)

四、陷阱：六個已排除的嫌疑，與一個仍未解開的差距

4.1 差距本身

指標	我們 (單卡 W7800)	AMD 官方卡	差
Wikitext word_perplexity	9.7297	8.8250	+10.25%
GSM8K flexible (兩次全量平均)	83.70%	91.51%	-7.81 pt
GSM8K strict (同上)	84.12%	90.37%	-6.25 pt

4.2 我們排除的 (附方法)

#	嫌疑	排除方法
1	權重量化	逐元素反量化比對 max 絕對差 = 0 (5 層) + 全層 496/496 + 對 BF16 源 $\cos \geq 0.9871$
2	LM head	PPL 以 echo 取 prefill logprobs, 而 INT4 LM head 只在 M ≤ 8 觸發 (日誌: <code>i4=1418 / miss=182</code>) ; 換 BF16 後 9.7297 → 9.7297 完全不變
3	量測儀器	echo 路徑與原生 <code>/generate</code> 對同 6 篇文件得 位元相同 (差 0.000%) ; token-PPL 5.52 × 1.32 = word-PPL 9.46 三數自洽
4	資料集	wiki-2 與 wiki-103 的 test split sha 相同 (62/62)
5	prompt 截斷	超長僅 0.61% 、缺答案格式僅 0.30%
6	我們自己的 K-split 優化	關掉 → 9.7296 (-0.00%)

⇒ 差距真實存在，機制我們尚未定位。我們不作猜測。

4.3 更重要的發現：我們自身的 greedy 解碼並不確定

上表兩次全量評測皆用 **greedy** (`do_sample=False`)，理應完全一致，實際卻相差 0.76 個百分點。

於是我們在同一部伺服器、同一配置之下，同樣 150 題連續執行兩次：

指標	結果
輸出完全相同	107 / 150 (71.3%)
對錯翻轉	3 題
兩次分數	85.33% 對 84.67%

⇒ 在我們的技術棧上，**greedy** 解碼是非確定的。

⇒ 因此我們不報告單一數字，而報告兩次全量評測的平均值，並標明自身噪音約 ± 1 pt。

這也是本文的一項關鍵提醒：任何「更動一個參數即可令分數上升 0.5 pt」的結論，在此噪音水平之下皆不可信。

我們懷疑來源是 split-K 的 **CAS 原子累加** (加法次序不固定)，或 LM head 低位寬路徑的 **M ≤ 8 閘** (精度隨 batch 大小跳變)。兩者皆未驗證。

4.4 幾項實用陷阱 (可為他人節省時間)

陷阱	內容
lm-eval 的 max_length 陷阱	API 後端有 <code>max_context_len = max_length - max_gen_toks</code> 。用預設 <code>max_length=2048</code> 配 AMD 的 <code>max_gen_toks=8192</code> ⇒ -6144 ⇒ prompt 截成空 ⇒ HTTP 400 (我們觸發了 26 45 次)。AMD 官方指令寫明 <code>`max_length=16384`</code> 。
LM head 的 M 閘	$M \leq 8$ 走 INT4、 $M > 8$ 走 BF16 ⇒ 同一 prompt 的 logits 會因並發批次的組成而異，屬可重現性風險
page-size 與 hybrid mamba	論壇實測： <code>page-size 64</code> 配 hybrid mamba 加 HiCache 本身即會段錯誤，必須 <code>page-size 1</code> (我們恰好為 1)
<code>`-gpureset`</code> 不可使用	對 gfx1100 會鎖住 PCIe root port，須冷開機 (社群警告)

五、經驗：我們學到的若干原則

- ① 先對證，後落手。凡涉及外部格式、協議、第三方語義，第一步必須尋求對證——而且優先閱讀官方源碼，不讀文檔描述。
- ② 驗證方法本身要先驗證。必須證明它能夠識別已知的壞輸入，而非僅止於「能夠運行」。(我們的一個精度閘門曾經對 NaN、全零、非有限輸出三個通道全部靜默判 PASS。)
- ③ 對證對象要驗身分。凡對證，必印被對證對象的 sha256。修改時間、路徑、檔名，一概不可信。
- ④ 硬件規格只准實測。同一張卡在四份材料中出現過 96 / 70 / 40 / 35 四個 CU 數，僅實測的 70 正確。
- ⑤ 質量集中處 ≠ 能量集中處。動手之前先量能量分佈。
- ⑥ 配件無辜，組合有罪。我們曾經把 HiCache 當作段錯誤主因；後來發現只有 DFLASH 與 HiCache 的組合才會出事，HiCache 本身無辜。
- ⑦ 一個中心為忠，兩個中心為患。量度之前先釐清有幾個中心；並行量測會互相污染，任何時候只准一個。

六、將來目標

#	目標	狀態
1	量測上游 vLLM 的新路線	上游已改為 M≤5 使用專用 INT4 skinny GEMM (`wvSplitK_int4_g` , wa ve 層分工 + DPP 歸約、無原子競爭)、M>5 使用 Triton。我們的 decode 有 80% 屬 M=4，恰好落在第一段。 A/B 工具已編譯完成，僅待執行
2	檢驗非確定性的機制	同一個 A/B 實驗同時可驗：CAS 原子 對 DPP 歸約
3	開啟 `--enable-deterministic-inference`	我們生產環境中此開關一直為 0。其說明為「batch invariant ops」，恰對應我們「精度隨 batch 跳變」的假設
4	將 kernel 層一併開源	目前僅發佈轉換器層；kernel 改動 (相對上游 18 commit / 42 檔) 仍在本地，尚未發佈
5	繼續追查那個 +10.25% / -7.81 pt 的差距	六個嫌疑已排除，機制尚未定位

七、致謝 —— 我們的技術層有 99% 來自他人

我們所做的那 1% 之所以能夠成立，是因為前面已經有人把路鋪好。逐項歸位如下：

引我們走上這條路的人

- Terry (lcz.me) —— 「你可以嘗試下 SGLang，論壇有帖子，體驗會好很多」。一句提示，令我們投入了十天。

我們的基礎：開源項目

- @StevenChenSE —— StevenChenSE/sglang 的 gfx1100-support 分支。我們整條生產線建在這上面。
- @vllm-project —— RDNA3 的 WMMA GPTQ kernel (我們拆解的那份 q_gemm_rdna3_wmma.cu，檔案頭即為 Copyright contributors to the vLLM project)
- @amd —— AMD Quark 量化工具鏈與 pack.py 的打包語義 (沒有它，我們的轉換器無從對證)
- @ExLlama 社群 —— GPTQ kernel 的血統

論壇前人 (我們沿其路線前行)

- @flyer666 —— lcz.me #1532：SGLang + HiCache 三級架構、為 HiCache 增加 RAM；#1640：gfx1100 單卡分析
- @抡锤者 / franklee006 —— lcz.me #1340 (4090 48G + DFlash2，DSH 寫碼均速 110 t/s)、#18288 (雙 7900 XTX 完整實戰)、#1329 (HiCache 實測)、#1587 (雙 4080S HiCache)
- @_折騰_ —— lcz.me #1500 / #1340：HiCache 多會話實戰、DFlash2 與 DSpark 的第一手主觀比較
- @Michael Zhou —— 4090D 48G 參數 (經 #1329 轉錄)：--mamba-full-memory-ratio 1.0、--mamba-scheduler-strategy extra_buffer、--mamba-track-interval 2048、page-size 1
- @清风明月 —— RTX PRO 4500 32G NVFP4 + HiCache 參數 (TID:1341)
- @Neo (lcz.me) —— HiCache L2/L3 原始帖
- @xiaote 老特 —— 參數調校與三方對照建議
- lcz.me #1252、#30599 系列 —— vLLM / SGLang 在 RDNA3 的早期實測

技術文章

- 智源 FlagOpen / FlagGems —— C++ Wrapper 與 Triton 算子呼叫鏈的實測文章 (小算子 wrapper 開銷 60–100 μs 的量級，與我們量到的每步固定開銷同階)

若有遺漏，敬請在下方指出 —— 這是我們的疏失，絕非本意。

八、技術附錄 (供其他 AI 與工程師取用)

本節供機器讀取，人類讀者可直接略過。

8.1 生產配置指紋

```
啟動腳本 sha256 : 7f52d3b0bfe876ad34d5c55c25e5e9a42b83afbae01b8a85891eac905f27a114
生產 .so sha256 : 7847e12eaf3d8b104f0227ab779dcfdef4248e843668d461334522598f2f5625
( 29,823,720 bytes = BLOCK_KN_SIZE 512 )
```

8.2 生效參數

```
--model-path ../Qwen3.8-27B-INT4-GPTQ-v3
--tp-size 1 --quantization gptq --dtype bfloat16 --mamba-ssm-dtype bfloat16
--kv-cache-dtype bf16 --attention-backend triton
--chunked-prefill-size 8192 --context-length 131072 --mem-fraction-static 0.88
--speculative-algorithm NEXTN ( 內部解析為 EAGLE / MTP-3 : steps 3 / topk 1 / draft 4 )
--cuda-graph-max-bs 32 --sleep-on-idle
--max-running-requests 4 --max-queued-requests 6
--enable-hierarchical-cache --hichache-ratio 1.0 --hichache-size 8
--hichache-write-policy write_through --hichache-io-backend kernel
--hichache-mem-layout page_first --enable-cache-report
--default-chat-template-kwarg {"enable_thinking": false}
( 無 --page-size => page_size = 1 )
```

8.3 環境變數 (全部，實讀自 /proc/pid/environ)

```
SGL_RDNA_CUSTOM_AR=0 SGL_RDNA_NO_FUSED=1 SGL_RDNA_GEMMA_TRITON=1
SGL_RDNA_VLLM_VERIFY=1 SGL_RDNA_LMHEAD_INT4=1 SGL_RDNA_LMHEAD_INT4_GS=128
SGL_RDNA_LMHEAD_INT8=0 SGL_WMMA_KSPLIT_AUTO=1
SGL_DTYPE=bfloat16 SGLANG_PHASE_TIMING=1 SGLANG_ENABLE_HEALTH_ENDPOINT_GENERATION=0
SGLANG_ENABLE_DETERMINISTIC_INFERENCE=0
SGLANG_MAMBA_SSM_DTYPE=bfloat16 SGLANG_CPROFILE_SPEC=0
TVM_FFI_DISABLE_TORCH_C_DLPACK=1
PYTHONPATH=../prod/rdna_shim
```

8.4 關鍵量測 (附儀器)

量	值	儀器
decode (DSH agent 路徑 ~20K ctx)	62.11 t/s	引擎 log gen throughput，running-req=1，n=11
decode (短提示單流)	85.05 t/s	同上
每步固定開銷 (gap > 20 μs)	2.4%	09-11 decode 段 817.12 ms 逐段
GPU busy (滿載)	89.2 – 94%	同兩次
GPU 滿載溫度 EDGE / HOTSPOT / MEM	79 / 99 / 94 °C	amd-smi (門檻 100/110/105)
節流	無 (GFX CLK 2148 = MAX_CLK)	amd-smi metric
每 token 有效權重頻寬	~385 GB/s (74 t/s × 19 GB ÷ accept 3.65)	推導

8.5 生產 kernel 的 ISA 拆解 (M≥16 WMMA 路徑)

反匯編自生產 .so 的 .hip_fatbin 內嵌 ELF，目標 gemm_q4_wmma_kernel_16x16_1w<__hip_bfloat16>：

指令	靜態條數
v_wmma	1
s_waitcnt	49
L lgkmcnt(0)	27
ds_load_u16_d16 + _hi	8 + 8 = 16 (對應源碼 for (i=0..15) b_frag[i] = b_tile[i][lane_lo])
ds_bpermute	8
global_atomic_cmpswap	8

編譯器 (AMD ROCm LLVM fork , clang 22.0.0git) 並非無能 —— 它在其他位置懂得使用 lgkmcnt(1)×98、(2)×20、(7)×24 等局部等待。是這段存取模式令它只能選擇最保守的方案。

8.6 上游 vLLM 的新路線 (我們尚未採用)

vllm/model_executor/kernels/linear/mixed_precision/rdna_hybrid_w4a16.py :

```
M <= MAX_SKINNY_BATCH_SIZE (=5) : HIP skinny GEMM (wvSplitK_int4_g)
M > MAX_SKINNY_BATCH_SIZE      : Triton W4A16 fused dequant GEMM
```

我們的 decode 有 80% 屬 M=4，恰好落在第一段；而我們目前兩段皆使用 GPTQ kernel。

8.7 硬體身份 (實測)

```
MARKET_NAME : AMD Radeon PRO W7800 48GB
SUBVENDOR_ID : 0x1458 (GIGABYTE)
DEVICE_ID : 0x7449 SUBSYSTEM_ID: 0x2428
NUM_COMPUTE_UNITS: 70 TARGET_GRAPHICS_VERSION: gfx1100
PCIE : Gen 4 x16 ( MAX_PCIE_WIDTH 16 / MAX_PCIE_SPEED 16 GT/s )
vBIOS : W7800 48G/F1/1133
```